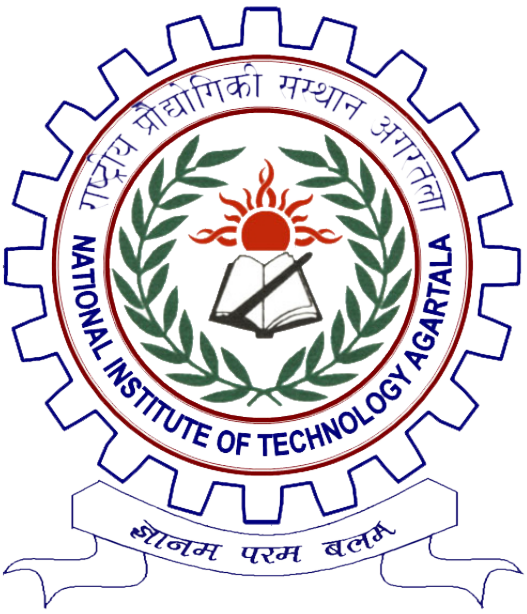
DIGITAL HORIZON CONNECTORS



Mainak Dasgupta (22UCS006)

COMPUTER SCIENCE & ENGINEERING DEPARTMENT
NATIONAL INSTITUTE OF TECHNOLOGY, AGARTALA
INDIA-799046
MAY, 2026

DIGITAL HORIZON CONNECTORS

Report submitted to
National Institute of Technology, Agartala
for the award of the degree
of
Bachelor of Technology

by
Mainak Dasgupta (22UCS006)

Under the Guidance of
Mr. Vivek Rodge
Software Development Manager, Amdocs

and

Dr. Awnish Kumar
Department Supervisor
Assistant Professor
CSE Department, NIT Agartala

COMPUTER SCIENCE & ENGINEERING DEPARTMENT
NATIONAL INSTITUTE OF TECHNOLOGY AGARTALA
MAY, 2026

Dedicated To

This report is dedicated to Prof. Mrinal Kanti Debbarma, HOD, CSE Department, NIT Agartala, my Project Manager Mr. Vivek Rodge, Software Development Manager, Amdocs and my Project Supervisor Dr. Awnish Kumar, Assistant Professor, CSE Department, NIT Agartala for sharing their valuable knowledge and encouragement and showing confidence on me all the time, to each of the faculties of the department who contributed to my development as a professional and helped me to achieve this goal, to my parents who supported me and believed in me and to all those people who have somehow contributed to the creation of this project.

❝First, solve the problem. Then, write the code.❞

- John Johnson

REPORT APPROVAL FOR B.TECH

This report entitled "Digital Horizon Connectors", by Mainak Dasgupta (22UCS006) is approved in partial fulfilment of the requirements for the award of Bachelor of Technology in Computer Science & Engineering.

Prof. Mrinal Kanti Debbarma	Dr. Awnish Kumar
(Head of the Department)	(Department Supervisor)
Professor	Assistant Professor
Computer Science and Engineering Department	Computer Science and Engineering Department
NIT Agartala	NIT Agartala

Mr. Vivek Rodge
(Project Manager)
Software Development Manager
Amdocs

Date:_____

Place: NIT, Agartala

DECLARATION

I declare that the work presented in this report titled "Digital Horizon Connectors", submitted to the Computer Science and Engineering Department, NIT Agartala, for the award of the Bachelor of Technology degree in Computer Science & Engineering, represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

MAY, 2026
Agartala

Mainak Dasgupta
(22UCS006)

CERTIFICATE

It is certified that the work contained in the dissertation titled 'Digital Horizon Connectors", by Mainak Dasgupta (22UCS006) have been carried out under our supervision. This work has not been submitted elsewhere for a degree.

Mr. Vivek Rodge
(Project Manager)
Software Development Manager
Amdocs

The dissertation titled "Digital Horizon Connectors", by Mainak Dasgupta bearing the Enrollment Number 22UCS006 is respectively endorsed for submission for the Bachelor of Technology in Computer Science & Engineering Department, NIT Agartala.

Dr. Awnish Kumar
Assistant Professor
(Department Supervisor)
Computer Science and Engineering Department
NIT Agartala

Acknowledgement

I take this opportunity to express my deep sense of gratitude to all who helped me directly or indirectly during this project work.
Firstly, I would like to thank my manager, Mr. Vivek Rodge, for being an exceptional guide and the best advisor I could ever have. His advice, encouragement and constructive feedback are sources of innovative ideas, inspiration and the causes behind the successful completion of this report. His confidence shown to me was the biggest source of inspiration for me. It has been a privilege working with him for the past 6 months.
I am highly obliged to all the faculty members of the Computer Science and Engineering Department for their support and encouragement. I also thank Prof.(Dr.) S. K. Patra, Director, NIT Agartala; Prof. Mrinal Kanti Debbarma, H.O.D, CSE Department; and Dr. Awnish Kumar, Assistant Professor, CSE Department, for providing excellent computing and other facilities, without which this work could not achieve its quality goal.

- Mainak Dasgupta (22UCS006)

List of Figures

List of Tables

Abstract

Enterprise API connector delivery in telecom environments is often fragmented across contract interpretation, bytecode inspection, manual field mapping, and script-based generation. This thesis presents a technical study of the API Development Tool, a web platform developed during an internship at Amdocs, which unifies this lifecycle in one guided workflow. The system combines a React frontend with a Node.js/Express backend to perform Swagger/OpenAPI parsing, JAR bytecode introspection for EJB metadata extraction, interactive mapping, YAML generation, and automated module creation with Socket.IO-based real-time feedback. Using source-code-level analysis and architectural evaluation, the study examines workflow coverage, parser behavior, persistence and synchronization correctness, security posture, and deployment readiness. Results show strong end-to-end workflow support and identify clear enhancement priorities in authentication, cross-platform generation, and automated testing.

Contents

CHAPTER 1

Introduction

Enterprise telecom integration teams repeatedly build API connectors that map external REST contracts to internal EJB-based services. The lifecycle is predictable but fragmented: engineers parse OpenAPI/Swagger files, inspect compiled JAR metadata, map request and response fields, prepare YAML configurations, and run generation scripts. In many teams, these steps are handled with separate tools and ad-hoc notes, which increases onboarding time, introduces inconsistency, and raises maintenance cost.

This thesis studies the API Development Tool built during an internship at Amdocs. The platform combines a React 18 frontend [4] and a Node.js/Express backend [3] to support specification upload, JAR bytecode introspection, interactive mapping, YAML generation, and connector module generation with Socket.IO [6] progress streaming. Its main contribution is workflow unification: previously disconnected activities become a single, traceable process.

1.1 Motivation

The original connector workflow relied heavily on manual interpretation and undocumented decisions. Engineers used text editors for API contracts, Java tools for class inspection, spreadsheets for mappings, and command-line scripts for generation. This approach consumed time and produced variation in naming, structure, and mapping quality.

Because downstream generation depends on strict YAML structure, mapping errors propagated into generated code and Maven artifacts. The process also lacked a central history of why mappings were chosen, making updates and debugging slow. These issues motivated a guided platform that enforces consistency and records decisions.

Table 1 summarises the transition from manual to tool-assisted development.

Table 1: Comparison of manual and tool-assisted connector development.

Aspect	Manual Workflow	Tool-Assisted Workflow
Specification analysis	Text editors, manual reading	Automated parsing with reference resolution and schema flattening
Service discovery	JAR decompilation in Java IDEs	Browser-based bytecode introspection without JVM
Field mapping	Spreadsheets, ad-hoc notes	Interactive dropdowns with source inference
Configuration authoring	Hand-written YAML files	Automated YAML generation from mapping state
Code generation	Command-line script invocation	One-click generation with real-time streaming
Traceability	Undocumented, scattered files	Full audit trail in database
Consistency	Varies by engineer	Enforced by structured workflow

1.2 Problem Context

The platform operates in an existing enterprise environment, not a greenfield system. It must align with a mature digital connector module tree, Maven-based build plugins,

compiled client-kit JAR artifacts, and mixed Windows/Linux deployment conditions, including air-gapped production networks.

Accordingly, generated outputs must match fixed directory conventions, such as Swagger files under src/main/resources/swagger/ and configs under src/main/resources/configs/. The key challenge is end-to-end continuity: connecting ingestion, introspection, mapping, generation, and synchronisation without breaking enterprise constraints.

1.3 Objectives

This thesis provides a code-level and architecture-level analysis of the platform. It traces how inputs move through frontend, backend, filesystem, and generation layers from specification upload to deployable output.

It also evaluates runtime and deployment behaviour, including Tomcat-hosted frontend delivery, PM2-managed backend operation, certificate handling, and filesystem-centric persistence. The study reports evidence-backed findings across security, reliability, portability, and maintainability, and proposes prioritised improvements.

1.4 Method of Study

The methodology is repository-driven technical analysis. Backend evaluation covers route logic, persistence design, synchronisation routines, and generation orchestration. Frontend evaluation covers workflow components, configuration editing, and API communication.

The Swagger parser, JAR parser, and configuration generator are examined in depth: reference resolution and schema flattening, bytecode and signature parsing, EJB detection, field-path extraction, source inference, and multi-EJB mapping assembly. Conclusions are grounded in observed implementation behaviour.

1.5 Report Organisation

The remaining chapters are organised as follows:

2. Chapter 2: Purpose and Scope defines project intent, functional boundaries, and expected outcomes.

3. Chapter 3: Related Works positions the platform against API-first tooling, code generators, and workflow automation approaches.

4. Chapter 4: System Architecture and Design details frontend, backend, utilities, synchronisation, generation, and deployment design.

5. Chapter 5: Results and Analysis presents evidence-based evaluation across workflow coverage, architecture, reliability, security, and portability.

6. Chapter 6: Conclusion and Future Work summarises contributions and outlines a phased enhancement roadmap.

CHAPTER 2

Purpose and Scope

2.1 Purpose of the Project

Enterprise connector delivery in telecom teams was historically fragmented across manual specification reading, JAR inspection, mapping spreadsheets, YAML editing, and script execution. This process produced inconsistent outputs and weak traceability.

The API Development Tool addresses that gap by consolidating the full lifecycle in one guided web application. A React 18 frontend [4] drives the workflow, and a Node.js/Express backend [3] manages parsing, persistence, synchronisation, and generation orchestration. The purpose is operationally clear: shorten delivery time, standardise mapping outputs, and preserve an auditable record of all connector decisions.

2.2 Functional Scope

The system scope covers five sequential phases that transform input artifacts into a deployable connector module. Figure 2.1 shows the artifact flow.

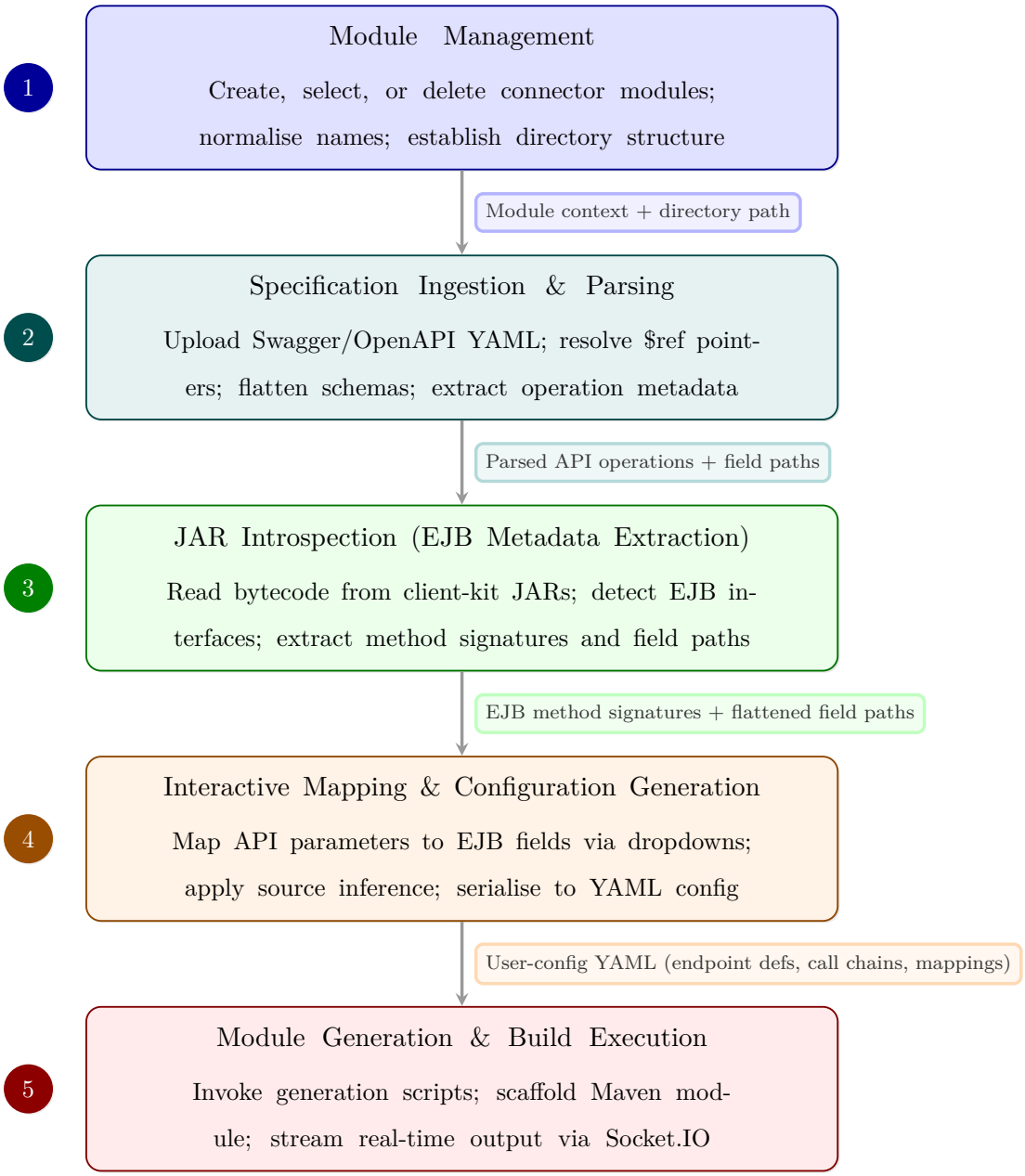


Figure 2.1: Five-phase workflow lifecycle of the API Development Tool, showing the key artifacts passed between each phase.

The phases are:

1. Module management: create/select/delete modules and enforce normalised names plus directory conventions.

2. Specification ingestion: upload Swagger/OpenAPI YAML, resolve references, flatten

schemas, and extract operation metadata.

3. JAR introspection: parse compiled client-kit bytecode, detect EJB interfaces, and extract method and field-path metadata with SHA-256 deduplication.

4. Interactive mapping & config generation: map request/response fields (including multi-EJB chains), apply source inference, and generate YAML.

5. Module generation: invoke the PowerShell-driven generation flow and stream execution output through Socket.IO.

2.3 Persistence and Synchronisation

State is maintained through a hybrid model: sql.js in-process SQLite [7] for metadata and filesystem storage for artifacts. The core tables are swagger_files, client_kit_metadata, and user_configs, with unique constraints for duplicate prevention.

Synchronization combines startup reconciliation, runtime chokidar watchers [2], and frontend polling fallback. Reconciliation repairs drift between disk and database; watchers detect external file changes; Socket.IO broadcasts keep clients aligned.

2.4 Non-Functional Considerations

The platform supports HTTPS/HTTP modes. In HTTPS mode, it auto-generates self-signed certificates at first startup, which fits air-gapped deployment constraints. The frontend is packaged as a Tomcat WAR [1]; the backend runs under PM2.

Portability is achieved through relative path resolution and dependency bundling. The stack runs without external database services or runtime internet access. A PowerShell packaging script automates validation, frontend build, WAR creation, and deployable ZIP assembly.

2.5 Future Enhancement Opportunities

The current scope covers the full connector workflow. Planned extensions include authentication and role control, cross-platform generation, stronger automated testing, containerised deployment, and multi-tenant isolation. The existing modular design can support these additions without architectural reset.

CHAPTER 3

Related Works

3.1 API-First and Contract-Driven Engineering

API-first engineering treats the API specification as the primary artifact for implementation, testing, and documentation [5]. OpenAPI/Swagger has become the standard contract format for machine-readable endpoint and schema definition.

In contract-driven practice, producer and consumer implementations must remain aligned with that specification. This is critical in enterprise integration, where REST APIs often front legacy EJB-based services. The platform in this thesis follows this model: it parses OpenAPI/Swagger contracts, derives structured metadata, and drives mapping and generation from that contract as the source of truth.

3.2 Existing Code Generation Ecosystems

OpenAPI Generator and Swagger Codegen show that specification-driven scaffolding can accelerate client and server development. They are effective for broad language coverage and template customization.

However, these generators are optimized for generic or greenfield outputs. Enterprise connector environments impose fixed module hierarchies, legacy EJB service contracts,

and organization-specific Maven pipelines. Template-based and model-driven approaches remain useful, but they often require substantial adaptation before they fit this context. The examined platform therefore applies a direct contract-to-enterprise transformation flow that combines parsing, metadata extraction, mapping, and generation in one pipeline.

3.3 Bytecode Introspection and Metadata Extraction

The platform's key differentiator is bytecode introspection of compiled client-kit JAR files without source code. Using java-class-tools, it parses class metadata in Node.js/browser environments and avoids JVM-only tooling.

Its extraction depth goes beyond signatures: it resolves hierarchy information, detects EJB interfaces through annotation and naming heuristics, parses generic signatures, flattens nested field paths, and normalizes synthetic parameter names. This output directly supports accurate interactive mapping.

3.4 Workflow Automation for Integration Teams

CI/CD systems automate downstream build and deployment, while low-code integration platforms automate high-level workflow design. Neither category directly addresses the full upstream connector-preparation workflow in legacy EJB environments.

The studied platform sits between these approaches. It offers guided mapping and automation comparable to low-code usability while preserving control required for organization-specific module structures and build chains.

3.5 Real-Time Communication in Development Tools

Real-time feedback is important for long-running generation tasks. Socket.IO [6] provides event messaging, reconnection, and fallback behaviour that fits enterprise network conditions.

In this platform, Socket.IO is used for (i) line-by-line generation log streaming and (ii) state synchronisation events for modules, specifications, client kits, and configurations, with polling fallback for resilience.

3.6 Comparison and Positioning

Existing tools cover fragments of the problem: generic generators focus on code templates, low-code tools focus on abstraction, and CI/CD tools focus on downstream automation. The studied platform combines contract parsing, bytecode extraction, interactive mapping, YAML generation, and scripted module creation in one enterprise-conformant workflow.

Its position is therefore distinct: compiled enterprise artifacts are first-class inputs, outputs match existing module conventions, and users receive real-time operational visibility.

Compared with generic generators, low-code platforms, and CI/CD pipelines, the platform studied here is distinctive in three combined capabilities: bytecode-driven enterprise metadata extraction, interactive contract-to-EJB mapping, and generation output aligned to existing module conventions in air-gapped deployment settings.

CHAPTER 4

System Architecture and Design

4.1 Overview of the System

The API Development Tool follows a two-tier client-server model with three functional layers: a React workflow frontend, a Node.js/Express orchestration backend, and an enterprise connector repository containing module structure, scripts, and Maven tooling. Communication is split between REST (CRUD and control operations) and Socket.IO (real-time logs and sync events). The backend links web interactions to filesystem artifacts and external generation scripts. Figure 4.1 shows this layered boundary.

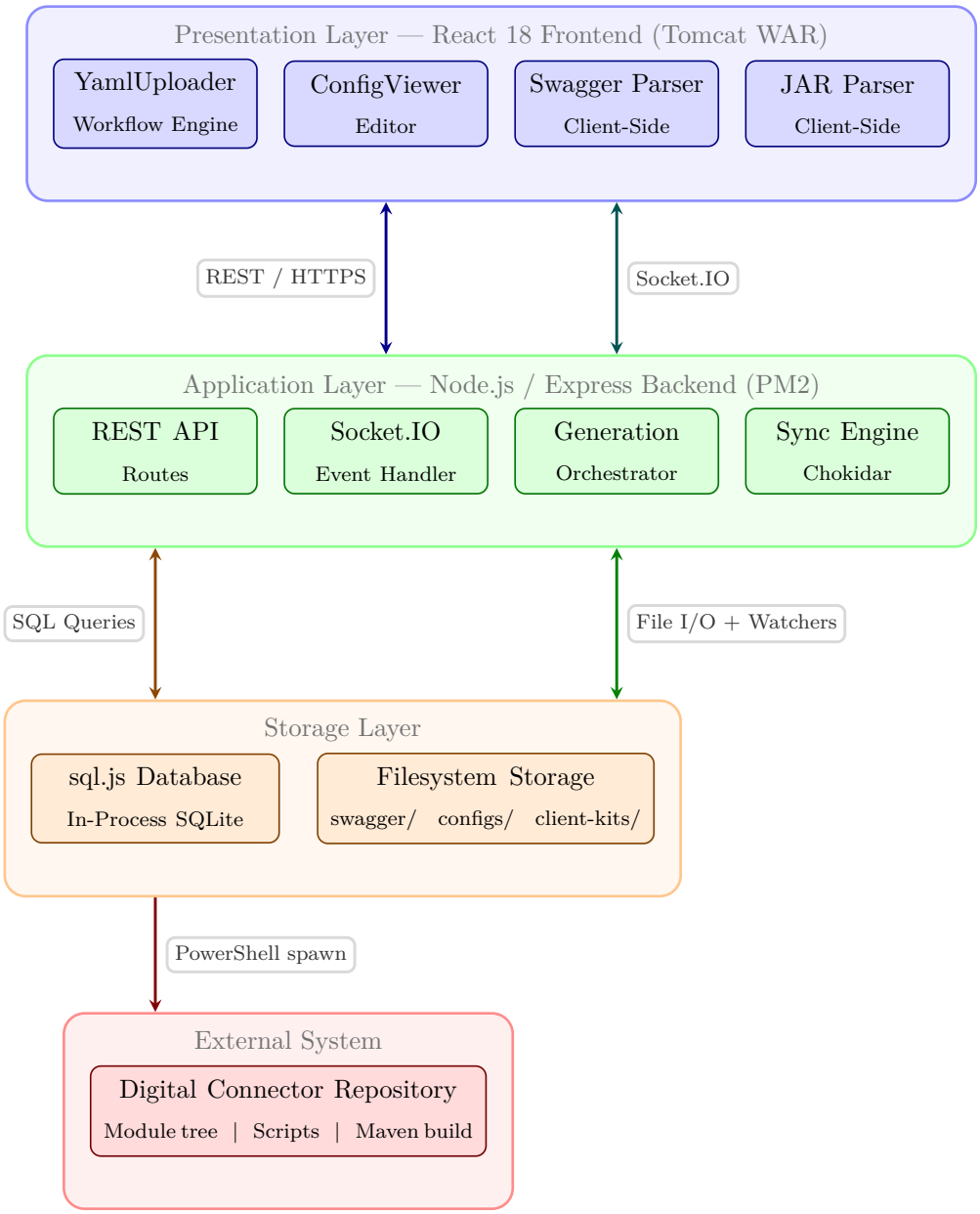


Figure 4.1: Layered system architecture of the API Development Tool, showing internal subcomponents, communication protocols, and deployment boundaries.

4.2 Frontend Architecture

The frontend is a React 18 [4] single-page app with React Router under the /retail base path. The root route hosts YamlUploader (primary workflow), while /configs hosts ConfigViewer (configuration review/editing).

YamlUploader centralizes workflow state for module context, specification data, JAR metadata, mappings, generation status, and console output. It establishes Socket.IO subscriptions and a polling fallback for environments with unstable WebSocket connectivity. Backend calls are wrapped in a unified API service with environment-driven base URL resolution. ConfigViewer reuses parser/generator utilities to keep editing behavior consistent with initial generation.

4.3 Utility Layer: Swagger Parser

The Swagger parser converts OpenAPI/Swagger documents into mapping-ready metadata through three linked steps: $ref resolution, schema flattening, and operation extraction.

$ref targets are resolved by traversing JSON pointer segments, with WeakMap-based caching to avoid repeated work. Flattening then produces deduplicated dot-path fields (including arrays and composed schemas via allOf/oneOf/anyOf) with circular-reference protection and depth bounds. Operation extraction combines these results to produce parameter and schema details per endpoint, including precedence-safe merge of path-level and operation-level parameters and support for both Swagger 2.0 and OpenAPI 3.0 request-body styles.

4.4 Utility Layer: JAR Parser and Bytecode Introspection

The JAR parser extracts EJB metadata directly from compiled archives without source code or JVM runtime requirements. Browser and server variants share the same core logic.

JAR files are read via jszip; class files are processed by java-class-tools in batches. The parser decodes JVM descriptors and generic signatures, resolves hierarchy details, and identifies service interfaces using javax.ejb.Remote/jakarta.ejb.Remote annotations plus naming heuristics.

Its critical output is deep, navigable field paths for method inputs and outputs. Recursive traversal includes inherited fields, getter-derived fields, and generic collection element types, bounded by configured depth and path caps for stability. Compiler-generated parameter names are normalized to meaningful names using type-based heuristics.

4.5 Utility Layer: Configuration Generator

The configuration generator converts interactive mappings into YAML required by the generation pipeline. It handles path parameters, request mappings, response mappings, and multi-EJB call ordering.

Request-side logic normalizes target paths (prefix cleanup, array notation, casing), supports source inference for unmapped fields, and rewrites interface-derived prefixes to implementation-compatible paths when needed. Response-side logic filters non-business fields, derives usable nested paths, and supports chained mappings across multiple EJB calls.

The output YAML includes module metadata, generation flags, endpoint definitions, EJB call sequences, and field correspondences, serialized through js-yaml.

The end-to-end transformation is linear and implementation-bound: Swagger/OpenAPI parsing and JAR introspection produce field metadata, interactive mapping resolves source-target correspondences, and the resulting YAML feeds module generation.

4.6 Backend Architecture

The backend is built on Express.js [3] and combines routing, storage, synchronization, real-time messaging, and generation orchestration in one service. Startup initializes environment variables, storage paths, directories, sql.js schema/migrations, reconciliation routines, and HTTP/HTTPS server plus Socket.IO.

REST endpoints are grouped by domain: modules, Swagger files, client kits, user configs, and generation, with a health endpoint for diagnostics. Metadata persistence uses sql.js [7] tables (swagger_files, client_kit_metadata, user_configs) with uniqueness constraints and synchronous export-on-write durability. Validation logic enforces file type and path safety (including path traversal checks) and normalizes module identifiers.

4.7 Filesystem Synchronisation

Consistency between disk state and metadata is maintained in two phases: startup reconciliation and runtime monitoring. Startup routines re-import missing files, remove stale records, and repair drift for modules, client kits, and configs.

At runtime, chokidar [2] watchers monitor module, client-kit, and config paths with debounced processing and sync coalescing. Confirmed changes trigger database updates and Socket.IO broadcasts (modules:sync, swaggers:sync, clientkit:sync, configs:sync). Generation events (generate:start, generate:output, generate:done) provide live execution visibility.

4.8 Module Generation Pipeline

Generation converts saved Swagger and user-config artifacts into deployable module code. The backend locates the generation script, invokes it through spawn, and supports safe-add (default), force, and optional skip-build modes.

Process output is streamed line-by-line via generate:output, and completion status is emitted through generate:done after success/failure marker checks. A ten-minute timeout protects against hung executions. On platforms without PowerShell, the endpoint returns HTTP 501 while leaving all non-generation features available.

4.9 Deployment Architecture

Deployment uses two services: frontend WAR on Tomcat [1] and backend Node.js service under PM2. Packaging is automated by a ten-step PowerShell flow covering dependency checks, frontend build, WAR creation, optional smoke tests, and final ZIP bundling.

All runtime paths derive from a configurable digital root, allowing the same codebase to run on Windows and Linux without hardcoded environment-specific paths.

4.10 Key Design Decisions

The implementation prioritizes self-contained deployment, operational transparency, and compatibility with existing enterprise tooling. Filesystem-centric artifact storage plus sql.js export-on-write persistence offers simple auditability and durability for team-scale use.

Workflow state centralization in YamlUploader favors end-to-end usability, while PowerShell-based generation preserves compatibility with established Windows-centric pipelines. Avoiding external database dependencies keeps deployment viable in air-gapped environments.

CHAPTER 5

Results and Analysis

5.1 Introduction

This chapter presents the technical evaluation of the API Development Tool based on source-code analysis, architectural review, and runtime behavior. The assessment covers workflow completeness, parser effectiveness, synchronization correctness, security posture, and deployment readiness.

5.2 Workflow and Parser Effectiveness

Backend routes and frontend flow confirm full lifecycle coverage across module management, specification ingestion, JAR introspection, mapping/configuration, and module generation.

Specification parsing supports OpenAPI/Swagger variants, resolves $ref, flattens composed schemas, and applies correct parameter-precedence rules. JAR parsing handles descriptor and generic-signature decoding, EJB detection by annotation and heuristics, and deep field-path extraction with bounded recursion. Configuration generation produces valid YAML for multi-EJB chains, request/response mappings, and interface-to-implementation path rewriting, with source inference reducing manual effort.

5.3 Architecture, Synchronisation, and Persistence

The two-tier architecture separates frontend workflow concerns from backend orchestration and persistence concerns. Backend endpoints are domain-grouped (modules, Swagger, client-kit, configs, generation), which reduces change impact.

Frontend centralization in YamlUploader provides coherent workflow control and remains decomposable into smaller hooks/components. Utility modules (Swagger parser, JAR parser, configuration generator) are reused across browser/server contexts, reducing behavior drift.

Realtime communication is effective for both generation logging and state synchronization. Socket.IO event streaming, reconnect behavior, startup state emission, debounced watcher handling, and polling fallback provide reliable eventual consistency in enterprise network conditions.

The sql.js layer correctly applies schema setup, migration, and uniqueness constraints. Duplicate prevention for Swagger (file+module) and client-kit content (SHA-256 hash) is enforced at storage level.

Startup reconciliation routines provide self-healing behavior by importing missing files, removing stale records, cascading dependent cleanup, and correcting path drift. Export-on-mutation persistence provides strong durability for the intended team-scale workload.

5.4 Security, Deployment, and Portability

The security model matches its intended trusted-network deployment. Endpoints are open by design, while upload and path handling apply extension checks and traversal-safe path resolution.

Generation requests are validated before script invocation. HTTPS can be enabled via auto-generated self-signed certificates. The architecture can be extended with authentication and stricter CORS if deployment scope expands.

Deployment has been validated in an air-gapped Linux environment, with path-resolution logic supporting both Windows and Linux file conventions. PM2 restart supervision and self-signed HTTPS support improve operational readiness.

Generation depends on PowerShell and therefore degrades gracefully on non-Windows platforms (HTTP 501 for generation while other features remain available). Packaging automation produces a self-contained deployable bundle with pre-check steps.

5.5 Consolidated Findings

The following table summarises the evaluation findings across the key dimensions assessed in this analysis.

Table 2: Consolidated findings matrix across evaluation dimensions.

Dimension	Assessment	Key Observation
Workflow Coverage	Strong	Complete end-to-end pipeline from specification upload through module generation
Architecture	Good	Clean separation between frontend workflow, backend orchestration, and utility layer
Parsing & Introspection	Strong	Comprehensive Swagger resolution and deep bytecode field path extraction
Real-time Feedback	Strong	Socket.IO streaming with debounced watchers and polling fallback
Persistence	Good	sql.js with startup reconciliation, hash-based deduplication, and self-healing sync
Security	Appropriate	Trusted-network design with path validation and HTTPS encryption
Portability	Good	Cross-platform path resolution with graceful environment adaptation
Testing	Extensible	Architecture supports progressive addition of test suites
Scalability	Fit for Purpose	Self-contained single-server model optimised for team-scale deployment

5.6 Performance Observations

Performance is appropriate for single-user or small-team operation. Swagger parsing is typically sub-second for common specs due to cached reference resolution.

JAR parsing is the dominant computation cost; batching, recursion bounds, and flatten-cache reuse control latency and memory growth. Generation time is mainly external (PowerShell + Maven), commonly tens of seconds to minutes, with real-time streaming reducing perceived wait and timeout guards preventing indefinite hangs.

CHAPTER 6

Conclusion and Future Work

6.1 Conclusion

This thesis presented a technical study of an enterprise API connector development platform and confirmed that the implemented workflow is complete, practical, and maintainable for its target environment.

The platform unifies five phases—module management, contract parsing, EJB metadata extraction, mapping/config generation, and module generation—that were previously executed through disconnected tools. The result is improved consistency, lower onboarding effort, and traceable mapping decisions.

Its architecture combines a guided React frontend, an Express orchestration backend, parser/generator utility modules, and real-time feedback via Socket.IO. The hybrid synchronization model (watchers + events + polling fallback) provides resilient behavior under enterprise network constraints. The evaluation also identifies a clear enhancement path for security, portability, storage throughput, and testing depth.

6.2 Key Contributions

The thesis contributes:

1. complete architectural documentation of frontend, backend, utility, storage, and generation layers;
2. algorithm-level analysis of Swagger parsing ($ref resolution, flattening, operation extraction);
3. detailed documentation of JAR bytecode introspection (descriptor/signature parsing, EJB detection, deep path extraction, parameter normalization);
4. evidence-based assessment across reliability, security, portability, and maintainability;
5. a prioritised roadmap for progressive production hardening.

6.3 Future Work

The platform is functionally mature, but further hardening is required for broader production use. Highest-priority work includes security controls, cross-platform generation support, storage evolution, and automated testing. Medium-priority work includes frontend refactoring and containerized deployment.

The roadmap is implemented as phased hardening:

- Security: introduce JWT/RBAC, stricter sanitization, and tighter CORS/certificate policies for broader deployment boundaries.
- Portability: replace or wrap PowerShell generation with a cross-platform orchestrator while preserving Windows compatibility during transition.
- Data and quality: evolve storage for higher concurrency and add unit, integration, and end-to-end automated tests in CI.
- Maintainability and scale: decompose YamlUploader, add containerized deployment, and prepare for tenant isolation and queued generation workloads.

These enhancements extend the same core architecture and are feasible as incremental releases. The workflow model is also transferable to other regulated integration domains where API-to-enterprise mapping is required and source-code access may be limited.

6.4 Summary

The thesis shows that a workflow-driven connector platform is both technically feasible and operationally valuable in enterprise integration settings. The evaluated implementation delivers strong coverage across parsing, introspection, mapping, generation, synchronization, and deployment.

The roadmap defines incremental next steps that build directly on the current architecture, enabling hardening and scale-up without fundamental redesign.

References

[1] Apache Software Foundation. Apache tomcat 9 documentation. https://tomcat.apache.org/tomcat-9.0-doc/, 2025. Accessed: April 2026.

[2] Chokidar Contributors. Chokidar file watching library. https://github.com/paulmillr/chokidar, 2025. Accessed: April 2026.

[3] Express.js Contributors. Express.js documentation. https://expressjs.com/, 2025. Accessed: April 2026.

[4] Meta Open Source. React documentation. https://react.dev/, 2025. Accessed: April 2026.

[5] OpenAPI Initiative. Openapi specification. https://spec.openapis.org/oas/latest.html, 2025. Accessed: April 2026.

[6] Socket.IO Contributors. Socket.io documentation v4. https://socket.io/docs/v4/, 2025. Accessed: April 2026.

[7] sql.js Contributors. sql.js documentation. https://sql.js.org/, 2025. Accessed: April 2026.

APPENDIX A

Biographical Sketch

Mainak Dasgupta

AD Nagar, Agartala, Tripura, PIN-799003,
E-Mail: dasguptamainak02@gmail.com, Contact. No. +91-9366415050

- Pursuing B.Tech. in Computer Sc. & Engg. branch from NIT Agartala with CGPA of 7.27.
- High School from Umakanta Academy (English Medium) under (T.B.S.E), Tripura with 87 % in 2019.
- Intermediate from Pranavanada Vidyamandir, under (C.B.S.E), Tripura with 77 % in 2021.